

A Study Report On Monarch Google's Planet-Scale In-Memory Time Series Database

Datta Ksheeraj
21CS30037

Getting into Context

What is Monitoring?

- Observing and Tracking

Why Monitoring?

- We care about our systems

To formally have nice definition, **monitoring** refers to the continuous process of observing and tracking the state, performance, and health of various systems, applications, and infrastructure components

At Google?

Until 2010 Google used **Borgmon** as its internal Monitoring system.

- It uses White box monitoring but later Black box monitoring was added

A Simple Example

- `% curl https://webserver:80/varz`

`http_requests 37`

`errors_total 12`

- `http_responses map:code 200:25 404:0 500:12`

What went wrong?

Google was expanding in terms of both products and geographical locations

So, Can't Google use Borgmon now?

- Decentralised architecture so have to develop a nice distributed system architecture
- Requires skill for the complexity it had

Other reasons mentioned in paper

- Semantic ambiguities
- No support for complex distribution types (Histogram)

Additionally..

Borgmon was actually excellent

- Some Googlers went to SoundCloud and created Prometheus based on this
- Now many open source monitoring systems do exist
- They use dashboards too for interface ..ex Graphana for Prometheus

Monarch

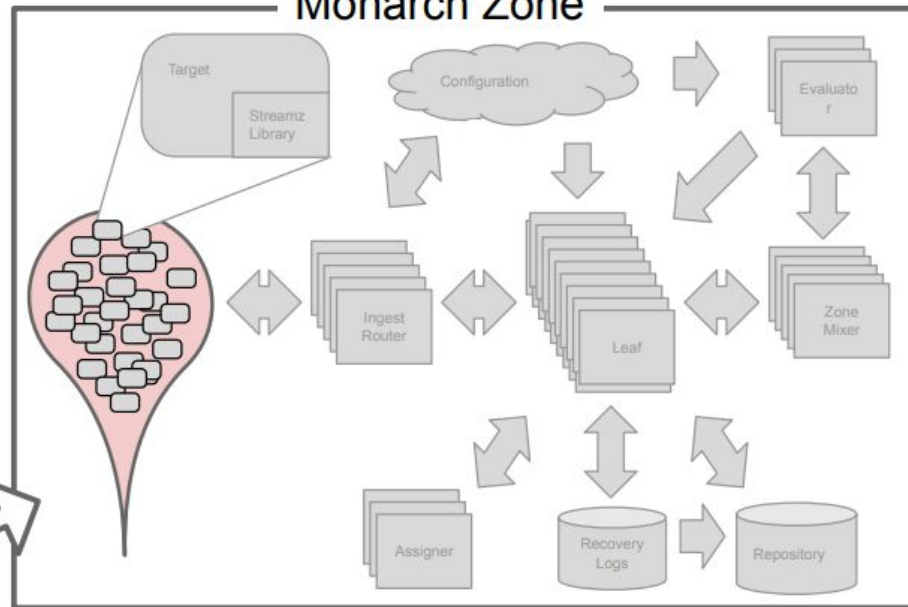
From 2010 it has been the Google's Monitoring system

Main Concern was to have a unified system simpler to use

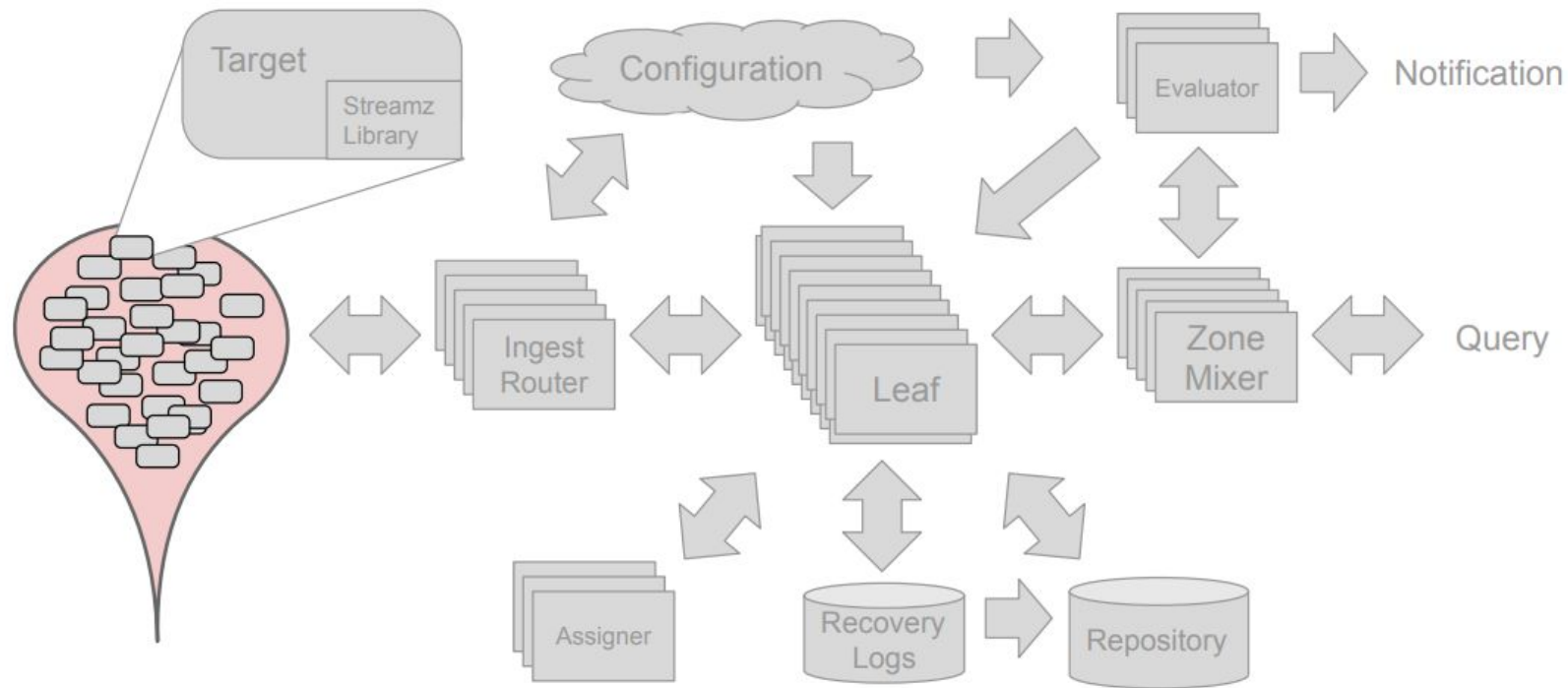
Monitor Locally



Monarch Zone



Monarch Zone: Ingestion, Retention and Queries



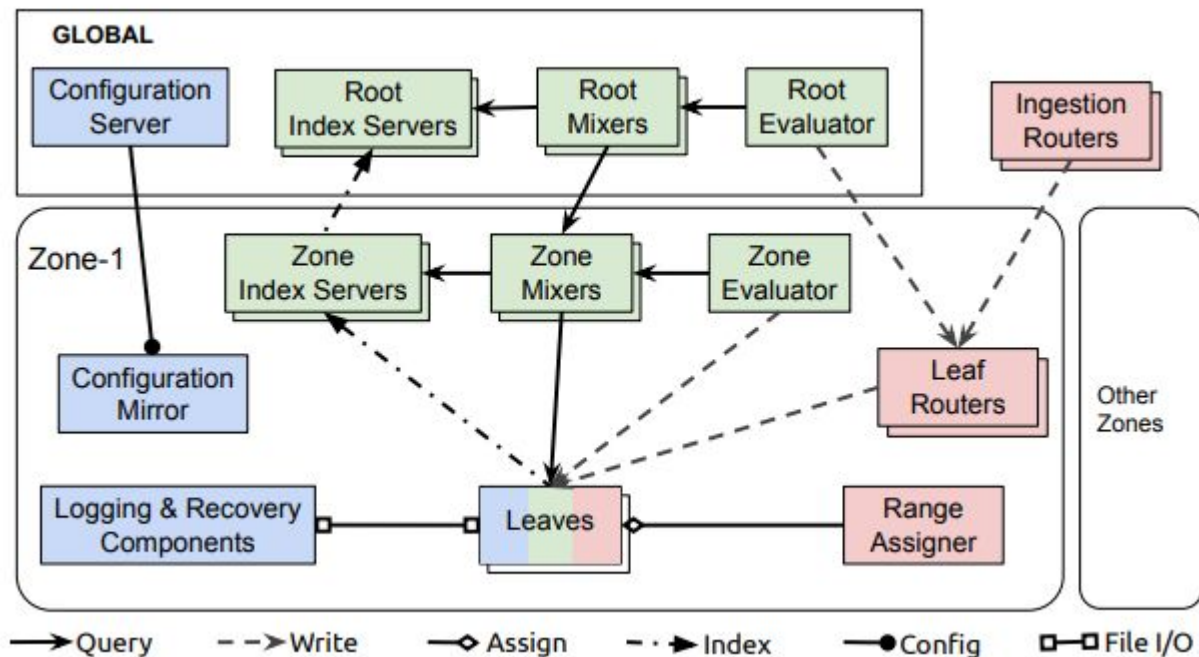
Components

State Holding

- Leaves
- Recovery Logs
- Configuration server

Data Ingestion

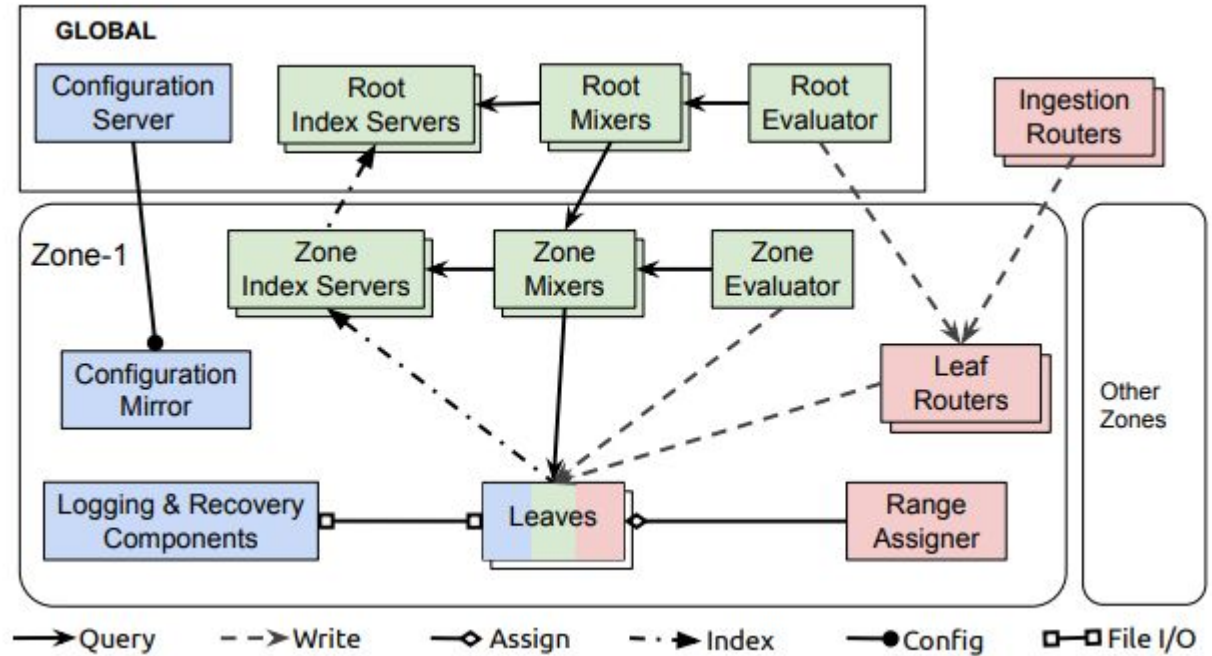
- Ingestion Routers
- Leaf Routers
- Range Assigners



Components

Query Execution

- Mixers
- Index Servers
- Evaluators



Before we proceed..

Monarch stores monitoring data as time series in schematized tables

Each row data consists of two columns namely

- Key columns
- Value columns

Again each Key column is divided into

- Targets
- Metrics

How Data reaches the leaves?

Suppose we monitor some system or server or database anything (call it A)

We configure it according to our requirements

Whenever A needs to send data to Monarch it happens in a sequence

- A sends that data to nearest ingestion router
- Based on location field in key column it is sent to leaf router of nearest zone (mapping there with ingestion router)
- Leaf routers have range maps (updated by leaves themselves)
- They send to appropriate leaves
- Periodically or time configurable it is written to recovery logs

Talking about Range Assigners

Range Assigners balance load among leaves of the zone

Whenever there is a more load on a particular leaf, it redistribute the ranges

- It selects a destination leaf with light load
- The new leaf informs the leaf router that this is the new one assigned
- Both source and destination leaf will be collecting data for that range until destination leaf recovers the previous data for that range from recovery logs
- Henceforth the data of that range can be erased from source leaf

Encoding

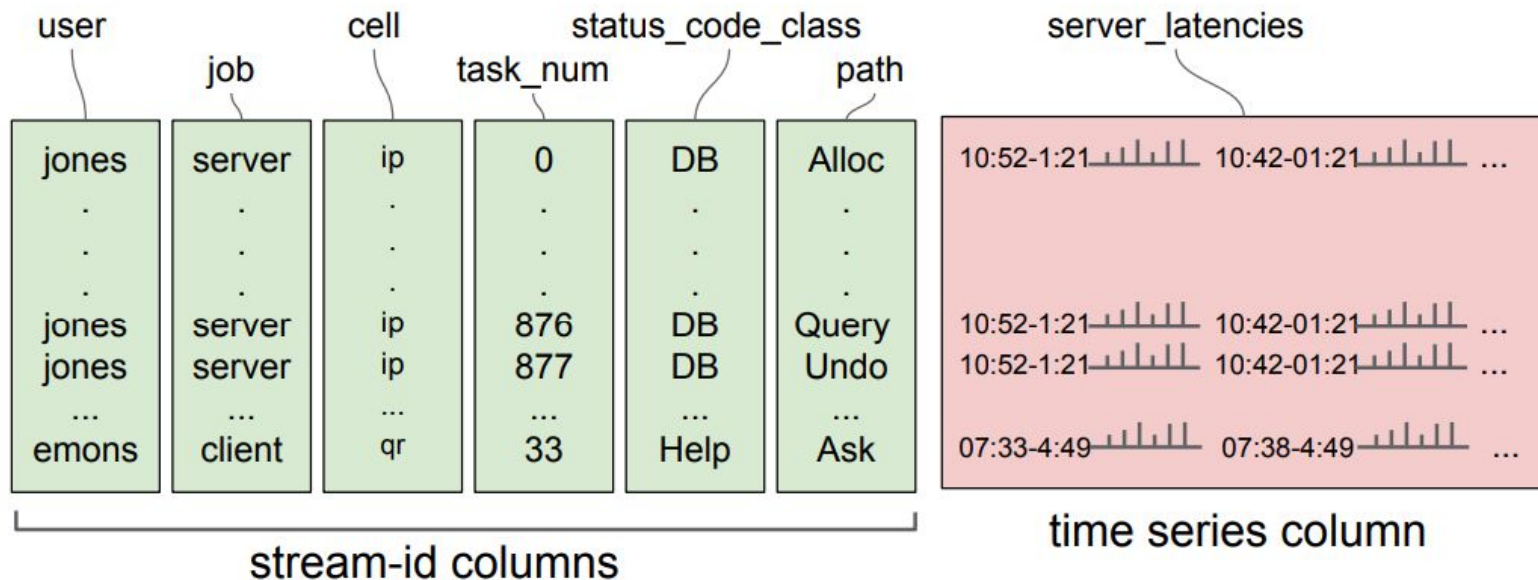
To save memory or disk space the data is compressed using some encoding technique

One of these is delta encoding which is a popular method to store in less space

We just store deltas from starting time /value of the attribute

The Data Model for Queries

BorgTask :: /rpc/server/server_latencies



Query Execution

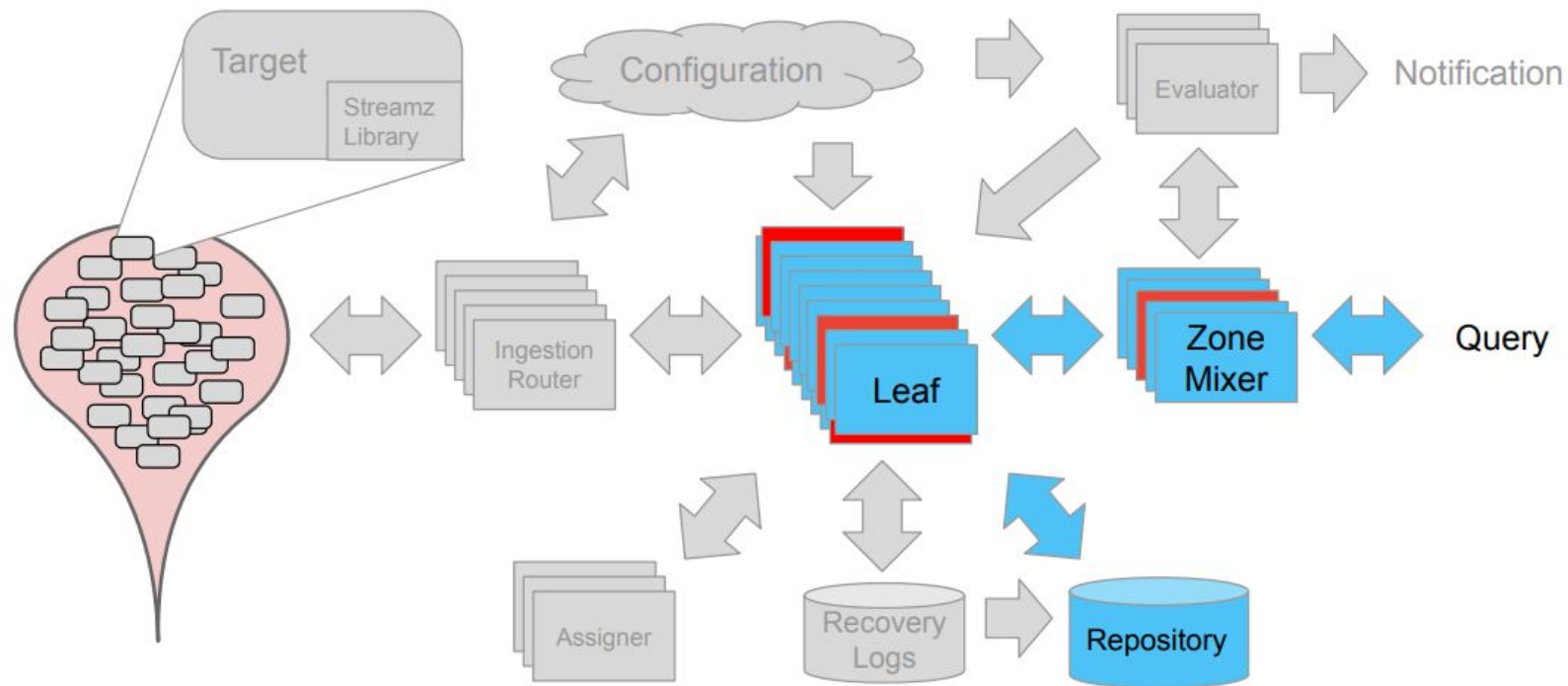
Two types of queries

- Standing queries
- Ad Hoc queries

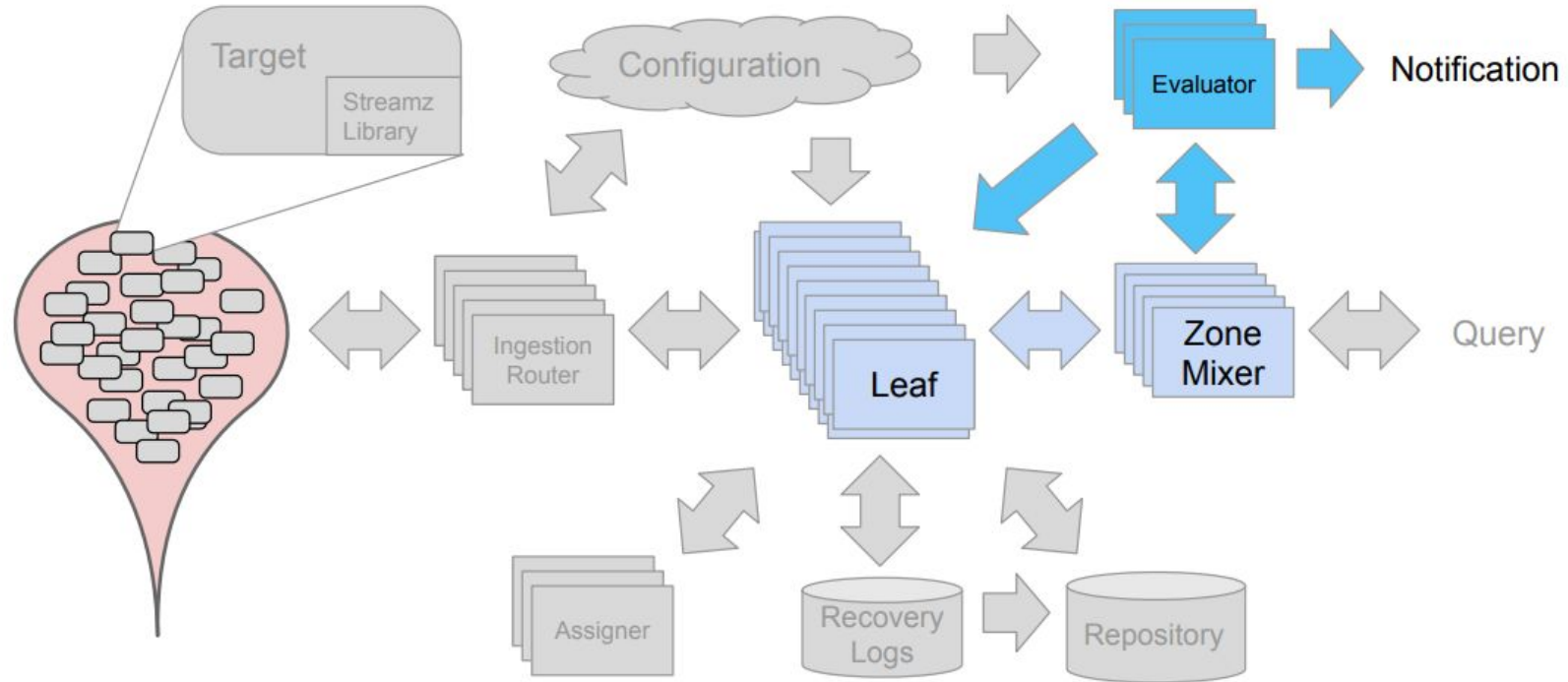
Query

```
table 'BorgTask' :: '/http/server/response_latency' |  
filter user = 'gmail' |  
group_by [job, cell], .percentile(95) | for 12h
```

Monarch Zone: Query



Monarch Zone : Evaluation and Notification



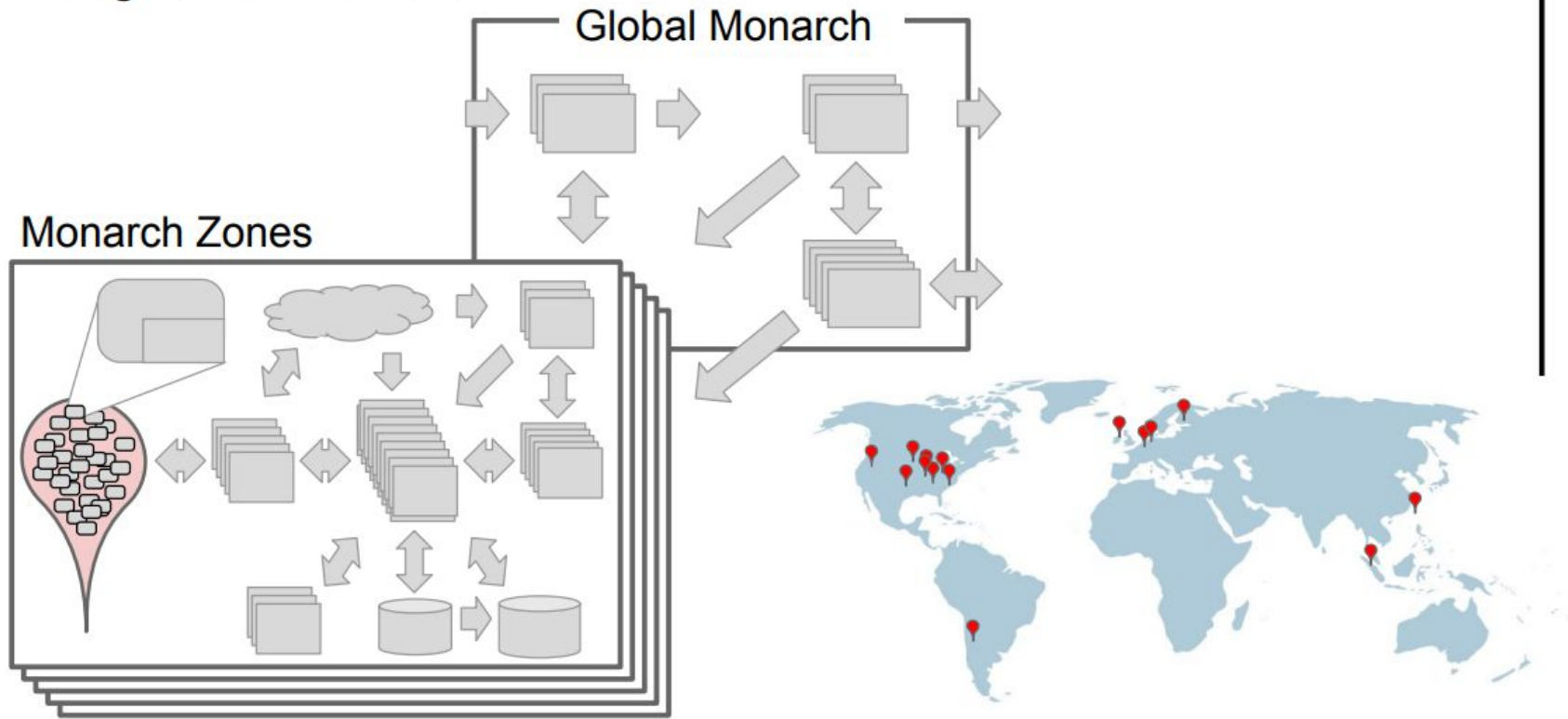
Getting into sequential steps...

- The root mixer gets the query
- It analyses the query and send to zoner mixers
- Zone mixers with the help of FHI (Field Hints Index) to leaves
- And now the result gets aggregated from the leaves to top

Replica Resolution

- Leaves mention the zone the quality of the info they are having
- Zone mixer finalises the info to be sent up

Integrated Monarch



Observations

It is mentioned in paper that

- Monarch readily trades consistency for high availability. For example Writing to or reading from a strongly consistent database like Spanner may block for a long time. So then if monarch has to use this database for some info it gets delayed (configuration file generally gets stored here)
- So it does stale reads and instead of waiting and informs the user about the quality of the info too.
- This might have been put up in accordance with use cases at google but may be we can use sort of RL to develop a model in order to learn those use cases making it more generalised

Just to Add..

Facebook's monitoring system is Gorilla, It was developed in a way with one of use case of making reads faster

Though not going into detail, its use cases are to deal with their own problems of their previous system

Even Microsoft has their own monitoring system

Interesting thing is these all tech companies established or updated their monitoring systems at similar time (around 2010-2014)

References

Some of pictures taken from

<https://www.datacouncil.ai/hubfs/Data%20Council/slides/nyc19/6.%20Monarch%20at%20Data%20Council%20NYC%2019.pdf?hsLang=en>

Colin Adams, Luis Alonso, Benjamin Atkin, John Banning, Sumeer Bhola, Rick Buskens, Ming Chen, Xi Chen, Yoo Chung, Qin Jia, Nick Sakharov, George Talbot, Adam Tart, and Nick Taylor. 2020. Monarch: Google's planet-scale in-memory time series database. Proc. VLDB Endow. 13, 12 (August 2020), 3181–3194. <https://doi.org/10.14778/3181-3194>